

Assignment 1 — Report

PCS1: Digital Design with Verilog HDL

Subject	PCS1
Assignment	Assignment 1
Name	Jayant Soni
Roll No.	B24CM1033

Introduction

This report presents the design, implementation, and verification of three fundamental digital logic modules using Verilog HDL: a 2-to-1 multiplexer, a 1-bit register with enable control, and a 1-bit Arithmetic Logic Unit (ALU). Each module is described at the Register Transfer Level (RTL) using synthesis-ready constructs, covering both combinational logic — where outputs depend only on current inputs — and sequential logic, which retains state across clock cycles.

Task 1: 2-to-1 Multiplexer

1.1 Design Overview

A 2-to-1 multiplexer is a combinational data selector that forwards one of two inputs to a single output based on a select line. It accepts inputs A and B, select signal sel, and produces output Y. When sel = 0, Y = A; when sel = 1, Y = B. This is captured by the Boolean expression: $Y = (\text{NOT sel}) \text{ AND } A \text{ OR sel AND } B$. MUXes are widely used in data routing, bus systems, and function generation circuits.

1.2 Implementation Methods

Method 1 — Continuous Assignment: A single assign statement with the ternary conditional operator evaluates continuously, updating the output immediately on any input change. This approach is concise, latch-free, and closely mirrors the Boolean equation, making it ideal for simple combinational expressions.

Method 2 — Procedural always Block: The selection logic is placed inside an always @(*) block with an if-else construct. The wildcard sensitivity list ensures re-evaluation on any input change. Y is declared as reg since it is driven from a procedural block, though it synthesises to purely combinational hardware.

1.3 Verification — Truth Table

sel	A	B	Expected Y	Observed Y
-----	---	---	------------	------------

0	0	0	0 (= A)	0
0	0	1	0 (= A)	0
0	1	0	1 (= A)	1
0	1	1	1 (= A)	1
1	0	0	0 (= B)	0
1	0	1	1 (= B)	1
1	1	0	0 (= B)	0
1	1	1	1 (= B)	1

Both implementations produced identical results for all eight input combinations. The output correctly followed A when sel = 0 and B when sel = 1, confirming functional equivalence and correct multiplexer behaviour.

Task 2: 1-Bit Register with Enable Control

2.1 Design Overview

A register is a sequential storage element that holds binary data across clock cycles. This module implements a 1-bit D-type flip-flop with an active-high synchronous enable. On every rising edge of clk, if en = 1, the register loads the value of D into Q. If en = 0, Q retains its previous value regardless of D. The enable check occurs synchronously, meaning it is evaluated only at the clock edge.

2.2 Role of the Enable Signal

The enable signal controls whether new data is accepted, using a feedback-MUX approach. When en = 0, Q is fed back to the flip-flop input, preserving the stored state. When en = 1, external data D is routed in instead. Inside the always @(posedge clk) block, non-blocking assignments (<=) are used to correctly model synchronous flip-flop behaviour and avoid simulation race conditions.

2.3 Verification — Timing Scenarios

Scenario	en	D	Q (before)	Q (after clk ↑)	Result
Load 0	1	0	1	0	Updated
Load 1	1	1	0	1	Updated
Hold (en=0)	0	1	0	0	Retained
Hold (en=0)	0	0	1	1	Retained

The register correctly loaded new data when $en = 1$ and retained its stored value when $en = 0$, confirming correct synchronous enable-controlled storage behaviour.

Task 3: 1-Bit Arithmetic Logic Unit (ALU)

3.1 Design Overview

An ALU is the computational core of a digital processor, responsible for performing arithmetic and logical operations. This module implements a 1-bit ALU supporting four operations selected by a 2-bit opcode. Inputs are A and B (1-bit each); output is Y. The operations are: AND (opcode 00), OR (opcode 01), XOR (opcode 10), and NOT A (opcode 11). NOT is a unary operation and ignores input B. A default case outputs logic 0 as a safe fallback for any undefined opcode.

3.2 Implementation Approach

The ALU uses an always @(*) block with a case statement to decode the opcode and route the appropriate operation to Y. The case construct is cleaner and more scalable than if-else chains for mutually exclusive selections. Each branch performs a direct bitwise operation: & (AND), | (OR), ^ (XOR), and ~ (NOT). Y is declared as reg, and the wildcard sensitivity list ensures fully combinational, clock-independent behaviour.

3.3 Verification — Operation Truth Table

X in column B denotes a don't-care, as the NOT operation does not use input B.

opcode	Operation	A	B	Expected Y	Observed Y
00	A AND B	0	0	0	0
00	A AND B	1	1	1	1
01	A OR B	0	1	1	1
01	A OR B	0	0	0	0
10	A XOR B	1	0	1	1
10	A XOR B	1	1	0	0
11	NOT A	0	X	1	1
11	NOT A	1	X	0	0

All four operations produced correct outputs. The NOT operation correctly inverted A while ignoring B, and the default case ensured no undefined output states.

Conclusion

This assignment demonstrated the design and verification of three core digital logic modules in Verilog HDL. The 2-to-1 MUX was implemented using both the assign statement and the always block, with both yielding identical results. The enable-controlled register illustrated synchronous sequential design using non-blocking assignments. The 1-bit ALU showed effective opcode decoding using the case construct. Together, the tasks reinforce RTL design principles and best practices for writing synthesisable, simulation-verified Verilog code.